

<https://github.com/xylokopa/Fy-Nance/blob/main/README.md>

⚠️ DOWNLOAD-HINWEIS

Download-Links NICHT einfach normal anklicken!

Linksklick nur wenn in der Tabelle angeboten ...und

Rechtsklick zum ECHTEN Download [ Download]-Link mit "Link speichern unter..." (bzw. "Ziel speichern unter...").

Nur so landen die Skripte sauber im richtigen UTF-8-Format in eurem z.B. `JupyterLab/Fy-Nance` Ordner!

⚠️ PREREQUISITES (Damit das Projekt lokal starten und auch offline laufen kann):

Lade in deinen jupyter lab-Ordner aus den Projekt-Komponenten MINDESTENS die ERSTEN 5 Dateien per RECHTE MAUSTASTE:

1] 02Apple_offline.csv

2] BOszi_Projekt.py

3] BOszi_Projekt_prerequisites.py

4] Oszi_03n_290626.ipynb

5] AAPL_Beispiel.ipynb

6] BOszi_03.pdf

Windows lokal : Python version 3 mindestens(3.14 reicht sicher) sollte installiert und per Windows-PATH eingetragen sein

PATH-Test : im Windows-Explorer Doppelklick auf die BOszi_Projekt_prerequisites.py => py.exe müsste starten und testen.

Notebook-Test : zB. mit `C:\jupyterlabs>jupyter lab` starten und `Oszi_03n_290626.ipynb` in Zelle mit Strg/Enter starten.

🔗 Fy-Nance: Projekt-Komponenten in Reihenfolge der Download-Präferenz

Komponente / Baustein	Typ	LINKE MAUSTASTE nur zum betrachten...	RECHTE MAUSTASTE Link speichern unter...	Beschreibung
02Apple_offline.csv	CSV			hier die Endung VON HAND auf .csv SETZEN !
BOszi_Projekt.py	PY			Hauptprogramm Python- Code
BOszi_Projekt_prerequisites.py	PY			Testprogramm Python- Code
Oszi_03n_290626.ipynb	iPYNB			jupyter Notebook Version 03n 29-06-26
AAPL_Beispiel.ipynb	iPYNB			Einführungsprogramm als Notebook
BOszi_03.pdf	PDF			Python-Script komplett- Doku PDF
POszi03_001	PNG			Screencopy when Running Program
PCA-Dokumentation	PDF			PCA-Ressources PDF
README.md schreib vor https	view- source:			hier die Endung VON HAND auf .md SETZEN !

```

1 # AAPL_Beispiel.py 50Zeilen Aktiensparen zwischen 2019 und 2021
2 import yfinance as yf
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import datetime
7 import os
8 csv_Nam = "02Apple_Offline.csv" # geht von 2015 bis 2026,boersentaeglich!
9 if os.path.exists(csv_Nam):
10     print(f"lokale csv {csv_Nam} wird geladen...")
11     df_csv = pd.read_csv(csv_Nam)
12     df_csv['Date'] = pd.to_datetime(df_csv['Date'])
13     df = df_csv.sort_values('Date').reset_index(drop=True)
14     df.set_index('Date', inplace=True)
15 df_aktiensparen = df.loc['2019-01-01':'2021-10-01'].copy()
16 df_monthly = df_aktiensparen.resample("MS").first()
17 df_monthly = df_monthly.dropna(subset=["Price"])
18 investment_per_month = 20
19 raten_plan = []
20 raten_nr = 0
21 for date, row in df_monthly.iterrows():
22     price = row["Price"]
23     units = investment_per_month / price
24     datum = date.strftime("%d.%m.%Y")
25     rate = round(units,6)
26     raten_nr = raten_nr + 1
27     print(raten_nr,datum,rate)
28     raten_plan.append((rate))
29 # depot-wert
30 gesamte_anteile = sum(raten_plan)
31 letzter_preis = df_monthly["Price"].iloc[-1]
32 depot_wert = gesamte_anteile * letzter_preis
33 # x-werte entspr. monatlich
34 x_zahlen = np.arange(len(df_monthly))
35 y = raten_plan
36 x_labels = [date.strftime("%d-%m-%y") for date in df_monthly.index]
37 plt.figure(figsize=(15, 6))
38 plt.plot(x_zahlen, y, label='Invest. 20€/mtl', color='red', linestyle='solid')
39 # x-Achse entspr Anzahl der Werte
40 plt.xlim(0, len(df_monthly) - 1)
41 # Anzeige jeder zweite Monat
42 plt.xticks(ticks=x_zahlen[::2], labels=x_labels[::2], rotation=0)
43 titel_text = (f"Aktien-Sparen mit AAPL (2019-2021)\n"
44             f"Gesamt-Anteile: {gesamte_anteile:.4f} Stück | "
45             f"Depot-Wert: {depot_wert:.2f} € (Kurs: {letzter_preis:.2f} €)")
46 plt.title(titel_text, fontsize=12, fontweight='bold')
47 plt.grid(True, linestyle='--', alpha=0.5)
48 plt.legend()
49 plt.tight_layout()
50 plt.show()

```

```

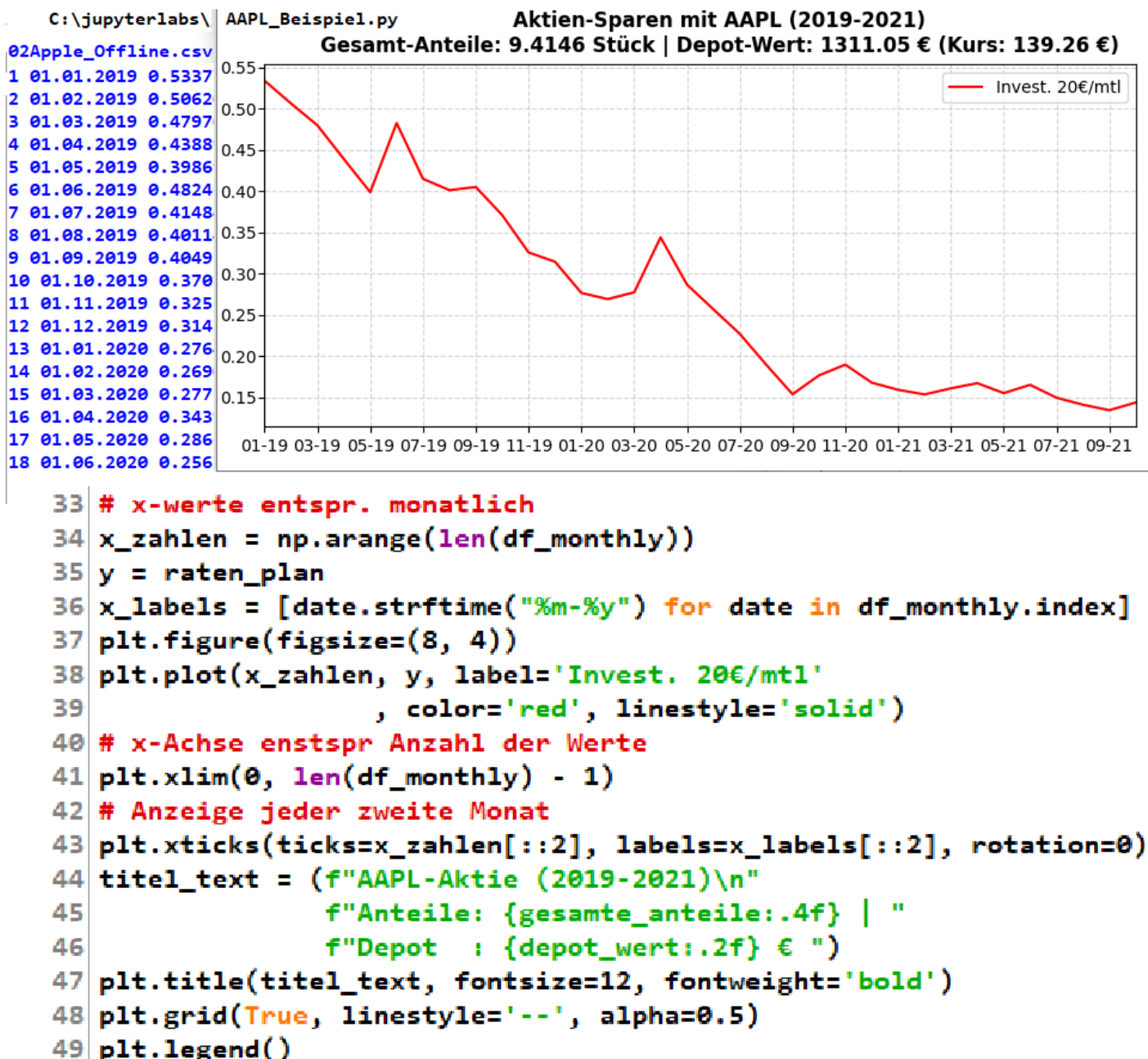
8 csv_Nam = "02Apple_Offline.csv" # geht von 2015 bis 2026
9 if os.path.exists(csv_Nam):
10     print(f"lokale csv {csv_Nam} wird geladen...")
11     df_csv = pd.read_csv(csv_Nam)
12     df_csv['Date'] = pd.to_datetime(df_csv['Date'])
13     df = df_csv.sort_values('Date').reset_index(drop=True)
14     df.set_index('Date', inplace=True)
15 df_aktiensparen = df.loc['2019-01-01':'2021-10-01'].copy()
16 df_monthly = df_aktiensparen.resample("MS").first()
17 df_monthly = df_monthly.dropna(subset=["Price"])

```

```

18 investment_per_month = 20
19 raten_plan = []
20 raten_nr = 0
21 for date, row in df_monthly.iterrows():
22     price = row["Price"]
23     units = investment_per_month / price
24     datum = date.strftime("%d.%m.%Y")
25     rate = round(units,6)
26     raten_nr = raten_nr + 1
27     print(raten_nr,datum,rate)
28     raten_plan.append((rate))
29 # depot-wert
30 gesamte_anteile = sum(raten_plan)
31 letzter_preis = df_monthly["Price"].iloc[-1]
32 depot_wert = gesamte_anteile * letzter_preis

```



```
BOszi_Projekt_prerequisites.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/Boersen-Oszi/BOszi_Projekt_pre...
File Edit Format Run Options Window Help
1 # prerequisites.py - REINER PREREQUISITES-TEST
2 # Version 03 R.Wu_GastH_Nr178854
3 # für alle nötigen importe
4 print("⌚ Prüfe System-Voraussetzungen... Bitte warten...\n")
5 try:
6     import math
7     import numpy as np
8     import pandas as pd
9     import yfinance as yf
10    from scipy import stats
11    import statsmodels.api as sm
12    import matplotlib.pyplot as plt
13    import matplotlib.dates as mdates
14    import matplotlib.ticker as mticker
15    from matplotlib.widgets import Slider, Button, TextBox
16    from datetime import datetime
17    import os
18    import csv
19    import locale
20    # Test, ob deutsche Formatierung auf dem PC
21    locale.setlocale(locale.LC_TIME, "de_DE")
22    print("=====")
23    print("✅ ALLES OK ! Die nötigen Pakete sind installiert.")
24    print("Das System ist bereit für den Börsen-Oszillographen.")
25    print("=====")
26 except ModuleNotFoundError as e:
27     # Fehlerfalle, sobald IDLE läuft
28     print("=====")
29     print(f"❌ FEHLER: Dir fehlt ein wichtiges Paket!")
30     print(f"    {e}")
31     print("\n🔧 REPARATUR-BEFEHL FÜR DEINE CMD:")
32     print("    pip install numpy pandas matplotlib scipy statsmodels yfinance")
33     print("=====")
34 except locale.Error:
35     print("⚠️ HINWEIS : deutsche Locale ('de_DE') nicht aktiv.")
36     print("    Skript läuft, aber Datumsformate können abweichen.")
37 input("\n[ENTER zum Schließen des Prerequisites-Tests]")
38
39
```

Ln: 36 Col: 4

as web-page

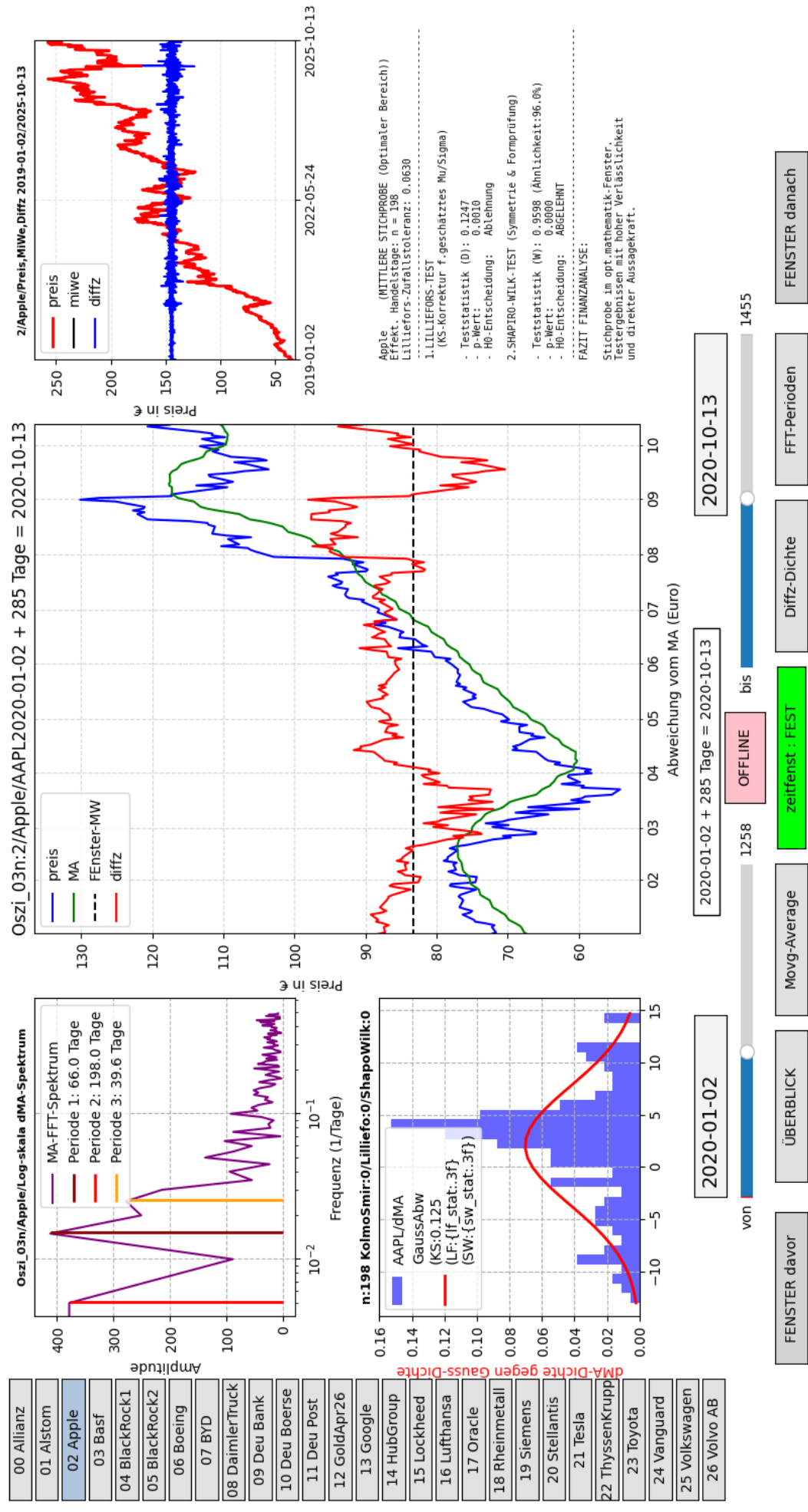
<https://xylokopa.github.io/Fy-Nance/>

as github-repository

<https://github.com/xylokopa/Fy-Nance>

BoersenOszi.py 16/28-06-2026 18h25 940.Z

Figure 1




```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
1 # BoersenOszi.py 16/28-06-2026 18h25 940.Z Version 03 R.Wu_GastH_Nr178854
2 # Alle Importe
3 import math
4 import numpy as np
5 import pandas as pd
6 import yfinance as yf
7 from scipy import stats
8 import statsmodels.api as sm # pip install statsmodels (Lilliefors)
9 import matplotlib.pyplot as plt
10 import matplotlib.dates as mdates
11 import matplotlib.ticker as mticker
12 from matplotlib.widgets import Slider, Button, TextBox
13 from datetime import datetime
14 import os
15 import csv
16 import locale
17 # FENSTER LAYOUT IN MOSAIC-TECHNIK
18 # %matplotlib widget
19 OSZ='Oszi_03n'
20 plt.rcParams['toolbar'] = 'None'
21 layout = [['.', '3Win', '1Win', '1Win', '4Win'], ['.', '2Win', '1Win', '1Win', '5Win'],
22           ['.', '.', '.', '.', '.']] # bei Lücke '.'
23 breitverh = [0.525] * 1 + [1.25] * 1 + [1] * 2 + [1.25] * 1
24 hoeheverh = [1] * 2 + [0.5] * 1
25 # Gesamtgröße not tight but constrained
26 fig,axd=plt.subplot_mosaic(layout,figsize=(15,8),
27                             gridspec_kw={'width_ratios':breitverh,'height_ratios':hoeheverh},
28                             layout='constrained')
29 # --- FENSTER ---
30 fig2 = fig
31 fig3 = fig
32 fig4 = fig
33 fig5 = fig
34 ax = axd['1Win']
35 ax.set_title('1.Fenster-Börsen-Oszillograph')
36 ax3 = axd['3Win']
37 ax2 = axd['2Win']
38 ax1 = axd['1Win']
39 ax4 = axd['4Win']
40 ax5 = axd['5Win']
41 # =====
42 #      0b      Importe und Lokalzeit über LOCALE aufrufen und speichern
43 # =====
44 datuhr='_Fig' # dummy wenn keine locale-Funktionalitaet
45 locale.setlocale(locale.LC_TIME, "de_DE")
46 datu=str(datetime.now())
47 datuhr=datu[2:4:1]+datu[5:7:1]+datu[8:10:1]+'_'+datu[11:13:1]+'h'+datu[14:16:1]
48 plt.rcParams['toolbar'] = 'None'
49 # =====
50 #      01      Projekt VARIABLEN
51 # =====
```

Ln: 657 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
51 # =====
52 val_online = 0          # default 1 = Online live, 0 = Offline
53 val_clr     = 1          # Default 1 = FEST(Zoom behalten), 0 = VAR (Reset)
54 akt_index   = 2          # Default-Ticker (Standard: 2 = Apple)
55 yfNAME = "AAPL"          # Default-Ticker für den Online-Start
56 offlinecsv = "02Apple_Offline.csv" # Default-Ticker für den Offline-Start
57 trigger = False          # start-stop zum Schaubild-zeichnen
58 ANF_DAT = "2015-01-02"    # Default-Fenster ANF
59 END_DAT = "2026-06-15"    # Default-Fenster END
60 moving_size = 20          # Moving-Average-Fenster
61 preis_linie = 1          # Preis-Linie ein
62 MAmiw_linie = 0          # MoveAverage aus
63 FEmiw_linie = 0          # Fenster-MiW aus
64 diffz_linie = 0          # PreisMA-Dif aus
65 FFT_Diagram = 0          # Fourier-Spektrum(Log) aus
66 line_preis = None
67 trigger = False
68 df = pd.DataFrame()      # globales DataFrame
69 # -----
70 #      02 Ticker-Liste aus CSV-Liste laden
71 # -----
72 try: ticker_df = pd.read_csv('Ticker-Liste_alt.csv')
73 except FileNotFoundError:
74     # Default Liste falls Ticker-Liste_alt.csv fehlt
75     ticker_liste = ""Num,Nam,Tik
76 00,Allianz,ALV.DE
77 01,Alstom,AOMD.DE
78 02,Apple,AAPL
79 03,Basf,BAS.DE
80 04,BlackRock1,BR
81 05,BlackRock2,BLK
82 06,Boeing,BA
83 07,BYD,002594.SZ
84 08,DaimlerTruck,DTG.DE
85 09,Deu Bank,DB
86 10,Deu Boerse,DBOY
87 11,Deu Post,DHL.DE
88 12,GoldApr26,GC=F
89 13,Google,GOOG
90 14,HubGroup,HUBG
91 15,Lockheed,LOM.F
92 16,Lufthansa,LHA.DE
93 17,Oracle,SOC.SG
94 18,Rheinmetall,RHM.DE
95 19,Siemens,SIE.DE
96 20,Stellantis,STLAM.MI
97 21,Tesla,TSLA
98 22,ThyssenKrupp,TKA.DE
99 23,Toyota,TOYOF
100 24,Vanguard,VWRL.SW
101 25,Volkswagen,VOW.DE
```

Ln: 657 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
101 25,Volkswagen,VOW.DE
102 26,Volvo AB,VOL1.SG""
103     with open('Ticker-Liste_alt.csv', 'w') as f:
104         f.write(ticker_liste)
105     ticker_df = pd.read_csv('Ticker-Liste_alt.csv')
106 # -----
107 #     03     zeitfenst-history anlegen
108 # -----
109 # GANZ OBEN INITIALISIEREN
110 history_bak = [] # Speicher für den "Linksknick" (Zurück)
111 history_vor = [] # Speicher für den "Rechtsknick" (Vorwärts)
112 def aktuellen_zustand_sichern():
113     # 1. Text-Strings direkt von der Matplotlib-Oberfläche
114     s_anf = text_box_start.text_disp.get_text().strip()
115     s_end = text_box_end.text_disp.get_text().strip()
116     # 2. Umwandlung in Timestamps (unabhängig von df-Länge)
117     dt_start = pd.to_datetime(s_anf)
118     dt_end = pd.to_datetime(s_end)
119     # 3. Tuple-Return Index und zeitfenst
120     return (akt_index, dt_start, dt_end)
121 # -----
122 #     04     Ticker-Daten Download oder aus csv lesen
123 # -----
124 def lade_ticker_daten(num_str, name, ticker_str):
125     global df, val_online # 1.globale Variablen
126     num0str = f"{int(num_str):02d}"
127     csv_name = f"{num_str}{name}_Offline.csv"
128     csv_nome = f"{num0str}{name}_Offline.bak"
129     # Download df_live mit 1.Spalte datum (von start bis end) und 2.Spalte Preis
130     if val_online > 0:
131         try:
132             print(f"Lade {ticker_str} live von Yahoo Finance...")
133             df_live = yf.Ticker(ticker_str).history(start=ANF_DAT, end=END_DAT)[["Close"]]
134             df_live.index = df_live.index.tz_localize(None)
135             df_csv = df_live.rename(columns={"Close": "Price"}).dropna().reset_index()
136             df_csv['Date'] = pd.to_datetime(df_csv['Date'])
137             df_csv.to_csv(csv_nome, index=False)
138             print(f"-> live geladen und in {csv_nome} gepuffert")
139             df = df_csv.sort_values('Date').reset_index(drop=True)
140             return
141         except Exception as e:
142             val_online = 0
143             print(f"{ticker_str} Download gescheitert ({e}) => Offline-csv")
144     # Offline df_csv 1.Spalte datum (von start bis end) und 2.Spalte Preis
145     if os.path.exists(csv_name):
146         print(f"lokale csv {csv_name} wird geladen...")
147         df_csv = pd.read_csv(csv_name)
148         df_csv['Date'] = pd.to_datetime(df_csv['Date'])
149         df = df_csv.sort_values('Date').reset_index(drop=True)
150     else:
151         print(f"Offline-csv {csv_name} nicht gefunden! Leeres Frame als Ersatz.")
```

Ln: 151 Col: 0


```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
151     print(f"Offline-csv {csv_name} nicht gefunden! Leeres Frame als Ersatz.")
152     df = pd.DataFrame(columns=['Date', 'Price'])
153     #-----
154     # 05 Default-Ticker Apple für erstes Schaubild
155     #-----
156     akt_ticker = ticker_df.iloc[akt_index] # Zeilenwahl per integer
157     lade_ticker_daten(f"{akt_ticker['Num']:02d}", akt_ticker['Nam'], akt_ticker['Tik'])
158     # -----
159     # 06 Achsen und Anheftungspunkte für Widgets platzieren
160     # -----
161     ax_von = fig.add_axes([0.21, 0.07, 0.22, 0.03]) # Anker von-Slider
162     ax_online = fig.add_axes([0.47, 0.062, 0.06, 0.05]) # Anker ON-OFFLINE-Schalter
163     ax_clr = fig.add_axes([0.44, 0.0125, 0.12, 0.035]) # Anker zeitfenst-Umschalt.
164     ax_bis = fig.add_axes([0.56, 0.07, 0.22, 0.03]) # Anker bis-Slider
165
166     ax_box_start = fig.add_axes([0.21, 0.11, 0.12, 0.04]) # Anker Li.Datumsfenster
167     ax_box_end = fig.add_axes([0.66, 0.11, 0.12, 0.04]) # Anker Re.Datumsfenster
168     text_mitte = fig.text(0.40, 0.13, '', bbox=dict(facecolor='white',
169                                                    alpha=1.0, edgecolor='black'))
170     ax1_box_stu = fig.add_axes([0.21, 0.115, 0.70, 0.02]) # Anker Mitt.Statuszeile
171
172     ax_diag0 = fig.add_axes([0.10, 0.01, 0.10, 0.04])
173     ax_diag1 = fig.add_axes([0.22, 0.01, 0.10, 0.04]) # Anker Diagnose-Schalter
174     ax_diag2 = fig.add_axes([0.33, 0.01, 0.10, 0.04])
175     ax_diag3 = fig.add_axes([0.57, 0.01, 0.10, 0.04])
176     ax_diag4 = fig.add_axes([0.68, 0.01, 0.10, 0.04])
177     ax_diag5 = fig.add_axes([0.80, 0.01, 0.10, 0.04])
178     widget_axes = [ax_von, ax_bis, ax1_box_stu ]
179     for w_ax in widget_axes: w_ax.set_axis_off()
180     # -----
181     # 07 Vorbereitung Ticker-Klavier am linken Rand
182     # -----
183     ticker_klavier = []
184     schalter_handles = [] # Hält die Schalter-Referenzen im Speicher aktiv
185     anzahl_ticker = len(ticker_df)
186     for i in range(anzahl_ticker):
187         zeil = ticker_df.iloc[i]
188         num_str = f"{zeil['Num']:02d}"
189         btn_label = f"{num_str} {zeil['Nam']}"
190         # Tasten-Anheftung iterativ von oben nach unten (0.95 bis 0.02) platziert
191         y_pos = 0.97 - (i * 0.033)
192         ax_tick = fig.add_axes([0.01, y_pos, 0.08, 0.028])
193
194         # Aktiven Button farblich hervorheben
195         btn_color = '#b0c4de' if i == akt_index else '#e1e1e1'
196         btn = Button(ax_tick, btn_label, color=btn_color)
197         btn.label.set_fontsize(10) # typische Schriftgröße für gute Lesbarkeit
198         # -----
199         ticker_klavier.append((btn, num_str, zeil['Nam'], zeil['Tik'], i))
200         # -----
201     schalter_handles.append(btn) # Schutz gegen Pythons Garbage Collector
```

Ln: 201 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
201 schalter_handles.append(btn) # Schutz gegen Pythons Garbage Collector
202 # -----
203 # 08 Widgets - platzieren
204 # -----
205 slider_von = Slider(ax_von, 'von', 0, len(df)-1, valinit=0, valfmt='%0.0f')
206 slider_bis = Slider(ax_bis, 'bis', 0, len(df)-1, valinit=len(df)-1, valfmt='%0.0f')
207 btn_online = Button(ax_online, 'ONLINE' if val_online else 'OFFLINE',
208 color='aqua' if val_online else 'pink')
209
210 text_box_start = TextBox(ax_box_start, '', initial='')
211 text_box_end = TextBox(ax_box_end, '', initial='')
212 text_box_start.text_disp.set_fontsize(14) # Fontgröße (z. B. 12 oder 14)
213 text_box_end.text_disp.set_fontsize(14)
214 # CLR-Schalter direkt unter Online-Schalter
215 # Variable für CLR-Zustand (1 = FEST / Zoom behalten, 0 = RESET / Vollansicht)
216 val_clr = 1
217 btn_clr = Button(ax_clr, 'zeitfenst : FEST', color='lime')
218 btn_clr.label.set_fontsize(10)
219 # --- PLATZIERUNG DIAGNOSE-BUTTONS ---
220 btn_diag0 = Button(ax_diag0, 'FENSTER davor', color='#d3d3d3')
221 btn_diag1 = Button(ax_diag1, 'ÜBERBLICK', color='#e0e0e0')
222 btn_diag2 = Button(ax_diag2, 'Movg-Average', color='#e0e0e0')
223 btn_diag3 = Button(ax_diag3, 'Diffz-Dichte', color='#e0e0e0')
224 btn_diag4 = Button(ax_diag4, 'FFT-Perioden', color='#e0e0e0')
225 btn_diag5 = Button(ax_diag5, 'FENSTER danach', color='#d3d3d3')
226 # -----
227 # 09 Schaubild-Achsen skalieren
228 # -----
229 def axen_skalierung(target_ax, df_zeitfen):
230 dt_start = pd.Timestamp(df_zeitfen['Date'].iloc[0])
231 dt_end = pd.Timestamp(df_zeitfen['Date'].iloc[-1])
232 target_ax.set_xlim(dt_start, dt_end)
233 target_ax.set_ylim(df_zeitfen['Price'].min()*0.95, df_zeitfen['Price'].max()*1.05)
234 # -----
235 # SELEKTIVE BEHANDLUNG EINZELNER KACHELN ax(i) zB hier (ax4 / '4Win')
236 # -----
237 if target_ax == ax4 or target_ax.get_label() in ['4Win', 'ax4']:
238 dt_mitte = dt_start + (dt_end - dt_start) / 2 # zeitliche Mitte für Skalenstriche
239 target_ax.xaxis.set_major_locator(mticker.FixedLocator([mdates.date2num(dt_start),
240 mdates.date2num(dt_mitte),
241 mdates.date2num(dt_end)]))
242 # Formatierungs-Filter für Datumsausgabe Start , Mitte und Ende
243 def spar_formatierer(x, pos):
244 def abst(x,y):return(abs(x-mdates.date2num(y)))
245 passt=(abst(x,dt_mitte)<1.0)or(abst(x,dt_start)<1.0)or(abst(x,dt_end)<1.0)
246 if passt:return pd.to_datetime(mdates.num2date(x)).strftime('%Y-%m-%d')
247 return '' # Rest unsichtbar
248 target_ax.xaxis.set_major_formatter(mticker.FuncFormatter(spar_formatierer))
249 target_ax.xaxis.set_minor_locator(mticker.NullLocator()) # Keine Monatmarken
250 target_ax.grid(True, which='major', linestyle='--', alpha=0.3)
251 plt.setp(target_ax.get_xticklabels(), rotation=0, ha='center', fontsize=9)
```

Ln: 201 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
251 plt.setp(target_ax.get_xticklabels(), rotation=0, ha='center', fontsize=9)
252 else:
253     # -----
254     # NORMALFALL bzw HAUPTFENSTER (ax)
255     # -----
256     tage_fenster = (dt_end - dt_start).days
257     target_ax.set_xlim(dt_start, dt_end)
258     target_ax.xaxis.set_major_locator(mdates.YearLocator())
259     target_ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
260     if tage_fenster < 730: # => Monats-Markierungen
261         target_ax.xaxis.set_minor_locator(mdates.MonthLocator())
262         target_ax.xaxis.set_minor_formatter(mdates.DateFormatter('%m'))
263         target_ax.grid(True, which='both', linestyle='--', alpha=0.5)
264     else:
265         target_ax.xaxis.set_minor_locator(mdates.MonthLocator())
266         target_ax.xaxis.set_minor_formatter(mticker.NullFormatter())
267         target_ax.grid(True, which='major', linestyle='--')
268     plt.setp(target_ax.get_xticklabels(), rotation=0, ha='center')
269 # -----
270 #      10                      zeitfenst auslesen
271 # -----
272 def zeitfenst():
273     id_von = int(slider_von.val)
274     id_bis = int(slider_bis.val)
275     # Slider sichern
276     if id_von >= len(df):
277         id_von = max(0, len(df) - 100) # letzte 100 Tage des neuen Tickers
278     if id_bis >= len(df):
279         id_bis = len(df) - 1
280     # Daten holen
281     df_zeitfenst = df.iloc[id_von:id_bis+1].copy()
282     # Daten per abstrakter Tupel-übergabe bündeln und weitergeben:
283     tupel1 = df.tail(10),0,len(df)-1,pd.Timestamp(df['Date'].iloc[-1])
284     tupel2 = pd.Timestamp(df['Date'].iloc[-1]), "", ""
285     if df_zeitfenst.empty: return tupel1 + tupel2
286     # Auslesen abschließen
287     dt_start = pd.Timestamp(df_zeitfenst['Date'].values[0])
288     dt_end = pd.Timestamp(df_zeitfenst['Date'].values[-1])
289     s_start = dt_start.strftime('%Y-%m-%d')
290     s_end = dt_end.strftime('%Y-%m-%d')
291     return df_zeitfenst, id_von, id_bis, dt_start, dt_end, s_start, s_end
292 # -----
293 #      11a                      Bewertung statistische Tests
294 # -----
295 def Bewertung_popup(n,lf_stat,lf_p,lf_Hyp,sw_stat,sw_p,sw_Hyp,Nam):
296     global fig5
297     # Grenzziehung & Fallunterscheidung für den Text
298     if n < 30:
299         sample_class = "KLEINE STICHPROBE"
300         grenzwert_text = "N/A"
301         kommentar = ("WARNUNG: Geringe statistische Aussagekraft!\n")
```

Ln: 201 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
301     kommentar = ("WARNUNG: Geringe statistische Aussagekraft!\n"
302                 "Verteilungstests bei n < 30 sehr ungenau.\n"
303                 "Primär visuelles Histogramm nutzen.")
304     elif 30 <= n <= 250:
305         sample_class = "MITTLERE STICHPROBE (Optimaler Bereich)"
306         lilliefors_critical = 0.886 / np.sqrt(n)
307         grenzwert_text = f"{lilliefors_critical:.4f}"
308         kommentar = ("Stichprobe im opt.mathematik-Fenster.\n"
309                     "Testergebnissen mit hoher Verlässlichkeit\n"
310                     "und direkter Aussagekraft.")
311     else: # n > 250 (Trifft bei Ihren n=283 zu)
312         sample_class = "GROSSE STICHPROBE (Hohe Sensitivität)"
313         lilliefors_critical = 0.886 / np.sqrt(n)
314         grenzwert_text = f"{lilliefors_critical:.4f}"
315         kommentar = (f"BEM: Bei n={n} greift das Schärfe-Paradoxon.\n"
316                     f"Test bestraft kleinste Abweichungen. Trotz Ablehnung\n"
317                     f"zeigt Shapiro-Wilk ({sw_stat*100:.1f}%), dass die dMA-Grundform\n"
318                     f"im Kernbereich stabil der Gauß-Kurve folgt.")
319     # Entscheidungen textlich aufbereiten
320     lf_Hyp = "Annahme (Normalverteilt)" if lf_p > 0.05 else "Ablehnung"
321     sw_Hyp = "Annahme (Normalverteilt)" if sw_p > 0.05 else "ABGELEHNT"
322     # --- 2. String-Zusammensetzung im Monospace-Look ---
323     report_text = (Nam)+(
324         f"      ({sample_class})      \n"
325         f"Effekt. Handelstage: n = {n}\n"
326         f"Lilliefors-Zufallstoleranz: {grenzwert_text}\n"
327         f"-----\n"
328         f"1.LILLIEFORS-TEST\n (KS-Korrektur f.geschätztes Mu/Sigma)\n\n"
329         f"- Teststatistik (D): {lf_stat:.4f}\n"
330         f"- p-Wert:           {lf_p:.4f}\n"
331         f"- H0-Entscheidung:   {lf_Hyp}\n\n"
332         f"2.SHAPIRO-WILK-TEST (Symmetrie & Formprüfung)\n\n"
333         f"- Teststatistik (W): {sw_stat:.4f} (Ähnlichkeit:{sw_stat*100:.1f}%) \n"
334         f"- p-Wert:           {sw_p:.4f}\n"
335         f"- H0-Entscheidung:   {sw_Hyp}\n"
336         f"-----\n"
337         f"FAZIT FINANZANALYSE:\n\n{komentar}")
338     # --- 3. Matplotlib Canvas Update ---
339     bbox_props = dict(boxstyle="square", facecolor="white", edgecolor="none",
340                       linewidth=0, alpha=1.0)
341     ax5.axis('off')
342     ax5.text(0.00,1,report_text,
343            transform=ax5.transAxes,
344            fontsize=7,
345            verticalalignment='top',
346            fontfamily='monospace', # Tabellen(Zeichenbreite konstant)
347            bbox=bbox_props)
348     fig5.canvas.draw_idle()
349     # -----
350     def Bewertung_konsole(n,sw_stat):
351         global diffdata
```

Ln: 201 Col: 0


```

351 global diffdata
352 # 1. Berechnung des kritischen Grenzwerts (nur gültig ab n > 30)
353 if n >= 30:
354     lilliefors_critical = 0.886 / np.sqrt(n)
355     grenzwert_text = f"{lilliefors_critical:.4f}"
356 else:
357     lilliefors_critical = None
358     grenzwert_text = "N/A (Tabelle nutzen)"
359
360 # 2. Fallunterscheidung und Text-Würdigung
361 if n < 30:
362     Proben_typ = "KLEINE STICHPROBE"
363     Kommentar = ("Sehr geringe stat.Aussagekraft. Verteilungstest bei n<30\n"
364                  "sehr ungenau. Nur visuelle Histogramm-Analyse sinnvoll.")
365 elif 30 <= n <= 250:
366     Proben_typ = "MITTLERE STICHPROBE (Optimaler Testbereich)"
367     Kommentar = ("Stichprobe im optimalen mathem. Fenster. Testergebnisse\n"
368                  "hohe Verlässlichkeit und Aussagekraft.")
369 else: # n > 250
370     Proben_typ = "GROSSE STICHPROBE (Vorsicht: Hohe Sensitivität)"
371     Kommentar = (f"HINWEIS: n={n} Schärfe-Paradoxon. Test bestraft kleinste\n"
372                  f"Abweichungen. Shapiro-Wilk ({sw_stat*100:.1f}%),\n"
373                  f"dMA-Grundform folgt im Kernbereich stabil der Gauß-Kurve")
374 # --- Ausgabe-Template (Ausschnitt für Konsole) ---
375 print("=" * 70)
376 print(f"BEWERTUNG Ticker-Linie {n}-Handelstage :\n Proben-Typ: {Proben_typ}")
377 print(f"Stichpr.Umfang n:{n}|Lilliefors-Zufallstoleranz: {grenzwert_text}")
378 print("=" * 70)
379 print(f"Kommentar:\n",Kommentar)
380 print("=" * 70)
381 # -----
382 #      11b                      Schaubild neu zeichnen
383 # -----
384 def neu_zeichnen(id_von, id_bis):
385     global line_preis
386     global trigger
387     global df, ticker_df          # 2.Dataframe & Ticker-liste
388     global akt_index, moving_size # 3.Tick-Index & MA-Fenster
389     global ANF_DAT, END_DAT, datuhr
390     ##### EVENTS OFF #####
391     for obj in all_widget_events:
392         # bei Komponente mit passendem Feature,direkt stumm schalten
393         if hasattr(obj, 'eventson'):
394             obj.eventson = False
395         elif isinstance(obj, tuple):
396             # bei Tupel(vektor), Tupel-Elemente einzeln stumm schalten
397             for sub_obj in obj:
398                 if hasattr(sub_obj, 'eventson'):
399                     sub_obj.eventson = False
400     #####
401 #-#

```



```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
401 #-#
402 try:
403     akt_ticker = ticker_df.iloc[akt_index] # Zeilenwahl per integer
404     akt_Num = akt_ticker['Num']
405     akt_Nam = akt_ticker['Nam']
406     akt_Tik = akt_ticker['Tik']
407     akt0Num = f"{int(akt_ticker['Num']):02d}"
408     csv_Nam = f"{akt0Num}-{akt_Nam}_Offline.csv"
409     if trigger or df.empty: return
410     #####
411     trigger = True
412     # --- Database-Erweiterungen Refresh -----
413     df['MA'] = df['Price'].rolling(window=moving_size).mean()
414     df['Diff'] = df['Price'] - df['MA']
415     # -----
416     # Erstes Fenster ax1 Ticker-Linie mit Mov'Average MA und Abweichg. dMA
417     # -----
418     # 1. Hauptfenster putzen , Überschrift und Justierung
419     # -----
420     # STATT ax.clear(): Mosaik-schonendes Putzen der Linien
421     for linie in list(ax.lines): linie.remove()
422     for text in list(ax.texts): text.remove()
423     ax.set_title(f"{OSZ}:{akt_Num}/{akt_Nam}/{akt_Tik}:{id_von} bis {id_bis}",
424                 fontsize=12, fontweight='bold')
425     # -----
426     # 2c. Index-Validierung der Slider
427     # -----
428     id_von = max(0, min(id_von, len(df)-1))
429     id_bis = max(0, min(id_bis, len(df)-1))
430     if id_von > id_bis: id_von = id_bis
431     slider_von.set_val(id_von)
432     slider_bis.set_val(id_bis)
433     # -----
434     # 3. Synchronisierung des zeitfensts
435     # -----
436     df_zeitfenst, id_von, id_bis, dt_start, dt_end, s_start, s_end = zeitfenst()
437     if not df_zeitfenst.empty:
438         tage_diff = (dt_end - dt_start).days
439         # axen_skalierung(ax, dt_start, dt_end, df_zeitfenst)
440         txt = f"{OSZ}:{akt_Num}/{akt_Nam}/{akt_Tik}"
441         ax.set_title(txt + f"{s_start} + {tage_diff} Tage = {s_end}")
442         text_mitte.set_text(f" {s_start} + {tage_diff} Tage = {s_end} ")
443         # -----
444         # 2b. zeitfenst-Aktualisierung Datums-TextBoxen
445         # -----
446         if text_box_start.text_disp.get_text() != s_start:
447             text_box_start.set_val(s_start)
448         if text_box_end.text_disp.get_text() != s_end:
449             text_box_end.set_val(s_end)
450         # -----
451         # 4. Erste Hauptlinie (... danach clippt Matplotlib das per xlim)
```

Ln: 402 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
451 # 4. Erste Hauptlinie (... danach clippt Matplotlib das per xlim)
452 # -----
453 if preis_linie:
454     line_preis, = ax.plot(df['Date'], df['Price'], label='preis', color='blue')
455     ##### Matplotlib clipping #####
456     x_min_num, x_max_num = ax.get_xlim()
457     dt_start_aktuell = mdates.num2date(x_min_num).replace(tzinfo=None)
458     dt_end_aktuell = mdates.num2date(x_max_num).replace(tzinfo=None)
459     df_fenster = df[(df['Date']>=dt_start_aktuell)&(df['Date']<=dt_end_aktuell)]
460     ma_mittelwert = df_fenster['MA'].mean()
461     y_mamiw = [ma_mittelwert] * len(df) # _fenster)
462     # -----
463     # 2. Zweite Hauptlinie
464     # -----
465     if MAmiw_linie:
466         line_MAmiw, = ax.plot(df['Date'], df['MA'], label='MA', color='green')
467     # -----
468     # 3. Dritte Hauptlinie
469     # -----
470     if FEmiw_linie:
471         line_FEmiw, = ax.plot(df['Date'], y_mamiw, label='Fenster-MW',
472                               color='black', linestyle='--')
473     # -----
474     # 4. Vierte Hauptlinie
475     # -----
476     if diffz_linie:
477         line_diff, = ax.plot(df['Date'], ma_mittelwert + df['Diff'], label='diffz',
478                               color='red')
479     ax.set_xlabel('Abweichung vom MA (Euro)')
480     ax.set_ylabel('Preis in €')
481     ax.grid(True, linestyle='--')
482     ax.legend(loc='upper left')
483     # -----
484     # Refresh der aktuellen Daten im 'df_zeitfenst'
485     # -----
486     diff_dropna = df_zeitfenst['Diff'].dropna() # <-- nur df_zeitfenst
487     diff_data = diff_dropna.values
488     # Kontrolle ob genug Datenpunkte im Slider-Refresh-Fenster vorhanden
489     if len(diff_data) > 5:
490         # Statistik auf Basis des synchronisierten Slider-Fensters berechnen
491         mu = np.mean(diff_data)
492         sigma = np.std(diff_data) # verzerrter Schätzer
493         sigmd = np.std(diff_data, ddof=1)
494         alpha = 0.05
495         n_stat = len(diff_data)
496         sqrtln = 1/math.sqrt(n_stat)
497         # Bewertung_popup(n_stat, lf_stat, lf_p, lf_Hyp, sw_stat, sw_p, sw_Hyp)
498         ks_stat, ks_p = stats.kstest(diff_data, 'norm', args=(mu, sigma))
499         ks_Hyp = 1 if ks_p > alpha else 0
500         lf_stat, lf_p = sm.stats.lilliefors(diff_data, dist='norm')
501         lf_Hyp = 1 if lf_p > alpha else 0
```

Ln: 402 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
501     lf_Hyp = 1 if lf_p > alpha else 0
502     sw_stat, sw_p = stats.shapiro(diff_data)
503     sw_Hyp = 1 if sw_p > alpha else 0
504     # Konsole gibt aktuelle Fenster-Werte und Kolmogorov-Smirnov-Abweichung aus:
505     # print(f"Fenster [{s_start} bis {s_end}] -> KS-Abweichung: {ks_stat:.4f}")
506     # print(f"Nullhypothese KolmSmi={ks_Hyp},Lillifo={ks_Hyp},ShapWil={sw_Hyp}")
507     if (diffz_linie):
508         # Nur bei Konsolenbetrieb
509         # Bewertung_konsole(n_stat,sw_stat)
510         Bewertung_popup(n_stat,lf_stat,lf_p,lf_Hyp,sw_stat,sw_p,sw_Hyp,akt_Nam)
511     # -----
512     # Zweites Fenster ax2 dMA-Histogramm und Vergl.Gauss-Verteilung
513     # -----
514     ax2.clear()
515     if MAmiw_linie:
516         count, bins, ignored = ax2.hist(diff_data, bins=30, density=True,
517                                         alpha=0.6, color='blue', label=f'{akt_Tik}/dMA')
518     if diffz_linie:
519         txt = f'n:{n_stat} KolmoSmi:{ks_Hyp}/'
520         ax2.set_title(txt + f'Lilliefo:{lf_Hyp}/ShapoWilk:{sw_Hyp}',
521                     fontsize=9, fontweight='bold')
522         gauss = stats.norm.pdf(bins, mu, sigma)
523         txt = f'GaussAbw\n(KS:{ks_stat:.3f})\n'
524         ax2.plot(bins, gauss, color='red', linewidth=2,
525                 label=txt + '(LF:{lf_stat:.3f})\n(SW:{sw_stat:.3f})')
526         ax2.set_ylabel('dMA-Dichte gegen Gauss-Dichte', color='red')
527         ax2.legend()
528         ax2.grid(True, linestyle='--')
529     # -----
530     # Drittes Fenster ax3 FFT-Analyse Abweichung dMA vom Gleit-Mittelwert MA
531     # -----
532     signal = diff_data
533     N = len(signal) ##
534     if N > 5:
535         fft_werte = np.fft.fft(signal)
536         frequenzen = np.fft.fftfreq(len(signal))
537         halbe = len(signal) // 2
538         freq = frequenzen[:halbe]
539         amp = np.abs(fft_werte[:halbe])
540         # -----
541         # Konsolen-Ausgabe zur FFT-Analyse der dMA-Zyklen vs. Gleitmittelwert MA
542         # -----
543         # 2. Top 3 "Resonanzen" (Formanten), Frequenz 0 (Gleichanteil) weglassen
544         sortierte_idx = np.argsort(amp[1:])[::-1] + 1
545         # print("\n--- FREQUENZREZEPT DES TICKERS (FORMANTEN) ---")
546         for i in range(min(3, len(sortierte_idx))):
547             idx = sortierte_idx[i]
548             # Zyklusdauer in Tagen (Kehrwert der Frequenz)
549             zyklus_tage = 1.0 / freq[idx] if freq[idx] > 0 else float('inf')
550             # Relative Amplitude ( der fft-werte )
551             amp_prozent = (amp[idx] / np.sum(amp)) * 100
```

Ln: 402 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
551     amp_prozent = (amp[idx] / np.sum(amp)) * 100
552     # bei Konsolenbetrieb
553     # print(f"Periode {i+1} (Resonanz): {zyklus_tage:.1f} Tage Takt "
554     #       f"| Amplitude: {amp_prozent:.1f}%")
555     dominant_idx = np.argmax(amp[1:]) + 1
556     dominanter_zyklus = 1 / freq[dominant_idx]
557     # print(f"Dominanter Zyklus im Fenster: {dominanter_zyklus:.1f} Tage")
558     # -----
559     # FFT-Schaubild der dMA-Zyklen im Kurs-Diagramm vs. Gleitmittelwert MA
560     # -----
561     ax3.clear()
562     if FFT_Diagram:
563         ax3.set_xscale('log')
564         ax3.set_title(f"{OSZ}/{akt_Nam}/Log-skala dMA-Spektrum",
565                     fontsize=8, fontweight='bold')
566         # Amplituden-Spektrum logarithmisch aufzeichnen
567         ax3.plot(freq, amp, color='purple', label='MA-FFT-Spektrum')
568         # Die Top 3 Resonanzen als vertikale Linien
569         farben = ['darkred', 'red', 'orange']
570         for i in range(min(3, len(sortierte_idx))):
571             idx = sortierte_idx[i]
572             f_freqz = freq[idx]
573             amp_freqz = amp[idx]
574             zyklus_tage = 1.0 / f_freqz if f_freqz > 0 else 0
575             ax3.vlines(f_freqz, 0, amp_freqz, colors=farben[i], linestyle='-',
576                      linewidth=2, label=f'Periode {i+1}: {zyklus_tage:.1f} Tage')
577         ax3.set_xlabel('Frequenz (1/Tage)')
578         ax3.set_ylabel('Amplitude')
579         ax3.legend(loc='upper right')
580         ax3.grid(True, linestyle='--')
581     #-#
582     except Exception as e:
583         # Falls drinnen etwas abstürzt, sieht man den Grund!
584         print(f"KRITISCHER FEHLER IM SCHLEIFEN-KOERPER: {e} ")
585     #-#
586     finally:
587         # -----
588         # drei Leinwaende synchron neu rendern
589         # -----
590         axen_skalierung(ax, df_zeitfenst)
591         fig.canvas.draw_idle()
592         ##### EVENTS ON #####
593         for obj in all_widget_events:
594             # Wenn Komponente mit passendem Feature, direkt stumm schalten
595             if hasattr(obj, 'eventson'):
596                 obj.eventson = True
597             elif isinstance(obj, tuple):
598                 # Falls Tupel(vektor), Einzelkomponenten einzeln stumm schalten
599                 for sub_obj in obj:
600                     if hasattr(sub_obj, 'eventson'):
601                         sub_obj.eventson = True
```

Ln: 402 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
601         sub_obj.eventson = True
602         #####
603         trigger = False
604         #-----
605         #      12      event-überwachung Ticker-Klavier am linken Rand
606         # -----
607     def ticker_klavier_klick(num_str, name, ticker_str, button_index):
608         def callback(event):
609             global akt_index, df
610             # HISTORY-ANKER: Alten Zustand auf den Stack BEVOR geladen wird!
611             zustand = aktuellen_zustand_sichern()
612             history_bak.append(zustand)
613             history_vor.clear() # Neuklick löscht Vorwärts-Verlauf
614             print(f"\n[KLAVIER] Taste gedrückt: {num_str} {name} ({ticker_str})")
615             # 1. Alte Slider-Werte sichern
616             id_von_alt = int(slider_von.val)
617             id_bis_alt = int(slider_bis.val)
618             # History-Puffer: Datum als formatierten String sichern
619             dt_startaltstr=pd.to_datetime(df['Date'].iloc[id_von_alt]).strftime('%Y-%m-%d')
620             dt_end_alt_str=pd.to_datetime(df['Date'].iloc[id_bis_alt]).strftime('%Y-%m-%d')
621             # 2. Tastenfarbe umschalten ...
622             for btn_obj, _, _, idx in ticker_klavier:
623                 if idx == button_index:
624                     btn_obj.color = '#b0c4de'
625                     btn_obj.ax.set_facecolor('#b0c4de')
626                 else:
627                     btn_obj.color = '#e1e1e1'
628                     btn_obj.ax.set_facecolor('#e1e1e1')
629             akt_index = button_index
630             # 3. Neuen Ticker laden
631             lade_ticker_daten(num_str, name, ticker_str)
632             if df.empty: return
633             # Max. Grenzen anpassen
634             max_index_neu = len(df) - 1
635             slider_von.valmax = max_index_neu
636             slider_bis.valmax = max_index_neu
637             # 4. SLIDER STUMMSCHALTEN
638             slider_von.eventson = False
639             slider_bis.eventson = False
640             # 5. NEUE Datumsspalte in einheitliches String-Format
641             neue_datums_ser = pd.to_datetime(df['Date']).dt.strftime('%Y-%m-%d')
642             neue_datums_liste = neue_datums_ser.tolist()
643             # Start-Datum im neuen Ticker suchen
644             if dt_startaltstr in neue_datums_liste:
645                 final_von = neue_datums_liste.index(dt_startaltstr)
646             else:# Start erster verfügbaren Tag (Index 0)
647                 final_von = 0
648             # End-Datum im neuen Ticker suchen
649             if dt_end_alt_str in neue_datums_liste:
650                 final_bis = neue_datums_liste.index(dt_end_alt_str)
651             else:
```

Ln: 402 Col: 0


```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
651     else:
652         # Enddatum ausserhalb Ticker , d.h: maximaler Index
653         final_bis = max_index_neu
654         # 6. ABSICHERUNG GEGEN FEHLER (Anfang >= Ende)
655         if final_von >= final_bis: # Notfall-Daten(evtl.Vollansicht)
656             final_von = 0
657             final_bis = max_index_neu
658         # Slider visuell kalendarisch korrekt
659         slider_von.set_val(final_von)
660         slider_bis.set_val(final_bis)
661         # 7. TEXTBOX-SYNCHRONISATION AUS INDIZES
662         s_start_neu = neue_datums_ser.iloc[final_von]
663         s_end_neu = neue_datums_ser.iloc[final_bis]
664         text_box_start.set_val(s_start_neu)
665         text_box_end.set_val(s_end_neu)
666         # Event-Bremse lösen
667         slider_von.eventson = True
668         slider_bis.eventson = True
669         # 8. FINALES ZEICHNEN
670         neu_zeichnen(final_von, final_bis)
671     return callback
672 # -----
673 #      13      Funktionalität der CLR-Schalters
674 # -----
675 def on_clr_clicked(event):
676     global val_clr
677     if val_clr > 0:
678         val_clr = 0
679         btn_clr.label.set_text("zeitfenst : VAR")
680         btn_clr.color = "pink"
681         btn_clr.ax.set_facecolor("pink")
682         # print(" Moduswechsel: zeitfenst (VAR).")
683     else:
684         val_clr = 1
685         btn_clr.label.set_text("zeitfenst : FEST")
686         btn_clr.color = "lime"
687         btn_clr.ax.set_facecolor("lime")
688         # print("Moduswechsel: zeitfenst (FEST).")
689     fig.canvas.draw_idle()
690 # -----
691 #      14      Funktionalität des ONLINE-Schalters
692 # -----
693 def on_online_clicked(event):
694     global line_preis
695     global val_online
696     # Modus umschalten
697     if val_online > 0:
698         val_online = 0
699         btn_online.label.set_text("OFFLINE")
700         btn_online.color = "pink"
701     else:
```

Ln: 402 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
701 else:
702     val_online = 1
703     btn_online.label.set_text("ONLINE")
704     btn_online.color = "aqua"
705 # Aktuelle Aktie im neuen Modus nachladen
706 zeil = ticker_df.iloc[akt_index]
707 lade_ticker_daten(f"{zeil['Num']:02d}", zeil['Nam'], zeil['Tik'])
708 if not df.empty:
709     line_preis.set_data(df['Date'], df['Price'])
710     neu_zeichnen(int(slidebar_von.val), int(slidebar_bis.val))
711 fig.canvas.draw_idle()
712 # -----
713 # 15 Aufruf 1 Diagnose-Schalter ÜBERBLICK
714 # -----
715 def diag1_click(event):
716     global line_preis
717     global trigger
718     global df, ticker_df # 2.Dataframe & Ticker-liste
719     global akt_index, moving_size # 3.Tick-Index & MA-Fenster
720     global ANF_DAT, END_DAT, datuhr
721     akt_ticker = ticker_df.iloc[akt_index] # Zeilenwahl Ticker-Liste per integer
722     akt_Num = akt_ticker['Num']
723     akt_Nam = akt_ticker['Nam']
724     akt_Tik = akt_ticker['Tik']
725     akt0Num = f"{int(akt_ticker['Num']):02d}"
726     csv_Nam = f"{akt0Num}-{akt_Nam}_Offline.csv"
727     df_zeitfenst,id_von,id_bis,dt_start,dt_end,s_start,s_end = zeitfenst()
728     if df_zeitfenst is None or df_zeitfenst.empty: return
729     axen_skalierung(ax4, df_zeitfenst)
730     miwe_val = df_zeitfenst['Price'].mean()
731     df_zeitfenst['miwe'] = miwe_val
732     df_zeitfenst['diffz'] = miwe_val + df_zeitfenst['Price'].diff().fillna(0)
733     for linie in list(ax4.lines): linie.remove()
734     for text in list(ax4.texts): text.remove()
735     ax4.plot(df_zeitfenst['Date'], df_zeitfenst['Price'],
736             color='red', linewidth=2, label='preis')
737     ax4.plot(df_zeitfenst['Date'], df_zeitfenst['miwe'],
738             color='black', label='miwe')
739     ax4.plot(df_zeitfenst['Date'], df_zeitfenst['diffz'],
740             color='blue', label='diffz')
741     ax4.set_title(f"{akt_Num}/{akt_Nam}/Preis,MiWe,Diffz {s_start}/{s_end}",
742                 fontsize=7, fontweight='bold')
743     ax4.set_ylabel('Preis in €')
744     ax4.legend(loc='upper left')
745     # Timestamp-Wert direkt im zeitfenst abholen
746     dt_start_gesichert= pd.Timestamp(df_zeitfenst['Date'].values[0])
747     dt_end_gesichert = pd.Timestamp(df_zeitfenst['Date'].values[-1])
748     axen_skalierung(ax4, df_zeitfenst)
749     fig.canvas.draw_idle() # Overlay-Fenster zu Diagnose 1 ausgeben
750     # plt.show()
751 # -----
```

Ln: 402 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
751 # -----
752 #      16          Fenster 5 Diagnose-Schalter Moving Average
753 # -----
754 def diag2_click(event):
755     global preis_linie # Preis-Linie Semaphore
756     global MAmiw_linie # MoveAverage Semaphore
757     global FEmiw_linie # Fenster-MiW Semaphore
758     global diffz_linie # PreisMA-Dif Semaphore
759     global FFT_Diagram # FFT-Spektrum(Log)
760     preis_linie = 1 # ein
761     if MAmiw_linie == 1: MAmiw_linie = 0
762     else: MAmiw_linie = 1
763     if FEmiw_linie == 1: FEmiw_linie = 0
764     else: FEmiw_linie = 1
765     diffz_linie = 0 # aus
766     FFT_Diagram = 0 # aus
767     df_zeitfenst,id_von,id_bis,dt_start,dt_end,s_start,s_end = zeitfenst()
768     status = 'Status:id_von= '+str(id_von)+' bis '+' id_bis= '+str(id_bis)
769     neu_zeichnen(id_von,id_bis)
770     # plt.show()
771 # -----
772 #      17          Aufruf 3 Diagnose Differenzen-Verteilung
773 # -----
774 def diag3_click(event):
775     global preis_linie # Preis-Linie Semaphore
776     global MAmiw_linie # MoveAverage Semaphore
777     global FEmiw_linie # Fenster-MiW Semaphore
778     global diffz_linie # PreisMA-Dif Semaphore
779     global FFT_Diagram # FFT-Spektrum(Log)
780     preis_linie = 1 # Preis-Linie ein
781     if diffz_linie == 1: diffz_linie = 0
782     else: diffz_linie = 1
783     MAmiw_linie = 1 # ein
784     FEmiw_linie = 1 # ein
785     FFT_Diagram = 0 # aus
786     df_zeitfenst,id_von,id_bis,dt_start,dt_end,s_start,s_end = zeitfenst()
787     status = 'Status:id_von= '+str(id_von)+' bis '+' id_bis= '+str(id_bis)
788     neu_zeichnen(id_von,id_bis)
789     # plt.show()
790 # -----
791 #      18          Aufruf 4 Diagnose-Fourier Analyse
792 # -----
793 def diag4_click(event):
794     global preis_linie # Preis-Linie Semaphore
795     global MAmiw_linie # MoveAverage Semaphore
796     global FEmiw_linie # Fenster-MiW Semaphore
797     global diffz_linie # PreisMA-Dif Semaphore
798     global FFT_Diagram # FFT-Spektrum(Log)
799     preis_linie = 1 # ein
800     MAmiw_linie = 1 # ein
801     FEmiw_linie = 1 # ein
```

Ln: 402 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
801 FEmiw_linie = 1      # ein
802 diffz_linie = 1      # ein
803 if FFT_Diagram == 1: FFT_Diagram = 0
804 else: FFT_Diagram = 1
805 df_zeitfenst,id_von,id_bis,dt_start,dt_end,s_start,s_end = zeitfenst()
806 status = 'Status:id_von= '+str(id_von)+' bis '+' id_bis= '+str(id_bis)
807 neu_zeichnen(id_von,id_bis)
808 # plt.show()
809 # -----
810 #      18      Aufruf 0 History-Tasten DATUM zurück DATUM vorwärts
811 # -----
812 def status_reset(ticker_idx, start_dt, end_dt):
813     global akt_index, df
814     akt_index = ticker_idx
815     # 1. Ticker-Daten aus der Liste holen
816     akt_ticker = ticker_df.iloc[akt_index]
817     # wandelt Nummer (egal ob 2, "2" oder "02") in zweistelligen String
818     num_formatiert = f"{int(akt_ticker['Num']):02d}"
819     # mit korrekt formatierter Nummer Daten laden
820     lade_ticker_daten(num_formatiert, akt_ticker['Nam'], akt_ticker['Tik'])
821     # Notbremse, falls Datei noch leer (Verhindert Crash!)
822     if df is None or df.empty:
823         print(f"keine Daten für {akt_ticker['Nam']}! Abbruch History-Sprung")
824     return
825     # -----
826     df['Date'] = pd.to_datetime(df['Date'])
827     max_index_neu = len(df) - 1
828     slider_von.valmax = max_index_neu
829     slider_bis.valmax = max_index_neu
830     slider_von.eventson = False
831     slider_bis.eventson = False
832     final_von = np.searchsorted(df['Date'].values, start_dt.to_numpy())
833     final_bis = np.searchsorted(df['Date'].values, end_dt.to_numpy())
834     final_von = max(0, min(int(final_von), max_index_neu))
835     final_bis = max(0, min(int(final_bis), max_index_neu))
836     if final_von >= final_bis:
837         final_von = max(0, final_bis - 100)
838     slider_von.set_val(final_von)
839     slider_bis.set_val(final_bis)
840     # Textboxen aktualisieren (Hier definiert)
841     s_anf_log = df['Date'].iloc[final_von].strftime('%Y-%m-%d')
842     s_end_log = df['Date'].iloc[final_bis].strftime('%Y-%m-%d')
843     text_box_start.set_val(s_anf_log)
844     text_box_end.set_val(s_end_log)
845     # Event-Bremse lösen
846     slider_von.eventson = True
847     slider_bis.eventson = True
848     # Klavier-Farben abgleichen
849     for btn_obj, _, _, idx in ticker_klavier:
850         bg_clr = '#b0c4de' if idx == akt_index else '#e1e1e1'
851         btn_obj.color = bg_clr
```

Ln: 402 Col: 0

```
BOszi_Projekt_printout.py - C:/jupyterlabs/github-com_xylokopa/Fy-Nance/BOszi_Projekt_printout.py (3.14.0)
File Edit Format Run Options Window Help
851     btn_obj.color = bg_clr
852     btn_obj.ax.set_facecolor(bg_clr)
853     # 4. DAS FINALE ZEICHNEN
854     neu_zeichnen(final_von, final_bis)
855     print(f"->History-Sprung zu {akt_ticker['Nam']}: {s_anf_log} bis {s_end_log}")
856     # -----
857     def navi_bak(event):
858         global akt_index, df
859         if not history_bak:
860             print("-> History Ende: Kein Zurück mehr möglich.")
861             return
862         # 1. Aktuellen Zustand für den Vorwärts-Button sichern
863         history_vor.append(aktuellen_zustand_sichern())
864         # 2. Letzten Zustand vom Stapel holen
865         ziel_ticker_idx, ziel_start_dt, ziel_end_dt = history_bak.pop()
866         # 3. System auf diesen Zustand zwingen
867         status_reset(ziel_ticker_idx, ziel_start_dt, ziel_end_dt)
868     # -----
869     def navi_vor(event):
870         global akt_index, df
871         if not history_vor:
872             print("-> History Ende: Kein Vorwärts mehr möglich.")
873             return
874         # 1. Aktuellen Zustand für den Zurück-Button sichern
875         history_bak.append(aktuellen_zustand_sichern())
876         # 2. Nächsten Zustand vom Stapel holen
877         ziel_ticker_idx, ziel_start_dt, ziel_end_dt = history_vor.pop()
878         # 3. System auf diesen Zustand zwingen
879         status_reset(ziel_ticker_idx, ziel_start_dt, ziel_end_dt)
880     # -----
881     # 19 Textbox-ENTER-Submit-Funktionalität für das zeitfenst per Datums-Eingabe
882     # -----
883     def on_submit_start(text_input):
884         try:
885             target_date = pd.to_datetime(text_input.strip(), format='%Y-%m-%d')
886             idx = (df['Date'] - target_date).abs().idxmin()
887             neu_zeichnen(idx, int(slider_bis.val))
888         except ValueError:
889             neu_zeichnen(int(slider_von.val), int(slider_bis.val))
890     def on_submit_end(text_input):
891         try:
892             target_date = pd.to_datetime(text_input.strip(), format='%Y-%m-%d')
893             idx = (df['Date'] - target_date).abs().idxmin()
894             neu_zeichnen(int(slider_von.val), idx)
895         except ValueError:
896             neu_zeichnen(int(slider_von.val), int(slider_bis.val))
897     # -----
898     # 20 Schleife zur event-überwachung aller Klaviertasten
899     # -----
900     for btn_obj, n_str, nam, tik, b_idx in ticker_klavier:
901         btn_obj.on_clicked(ticker_klavier_klick(n_str, nam, tik, b_idx))
```

Ln: 402 Col: 0


```

897 # -----
898 #      20      Schleife zur event-überwachung aller Klaviertasten
899 for btn_obj, n_str, nam, tik, b_idx in ticker_klavier:
900     btn_obj.on_clicked(ticker_klavier_klick(n_str, nam, tik, b_idx))
901 # -----
902 btn_clr.on_clicked(on_clr_clicked)    # clr-Schalter abrufen
903 schalter_handles.append(btn_clr)      # in aktive handles-Liste eintragen
904 btn_online.on_clicked(on_online_clicked) # online-Schalter abrufen
905 btn_diag0.on_clicked(navi_bak)        # History-Schalte zurück
906 btn_diag1.on_clicked(diag1_click)     # Diagnose-Schalter abrufen
907 btn_diag2.on_clicked(diag2_click)
908 btn_diag3.on_clicked(diag3_click)
909 btn_diag4.on_clicked(diag4_click)
910 btn_diag5.on_clicked(navi_vor)        # History-Schalte vorwärts
911 # -----
912 #      21      Huckepack-Aufruf der beiden Slider um beim Schieben plot neu zu zeichnen
913 # -----
914 slider_von.on_changed(lambda v:neu_zeichnen(int(slider_von.val),int(slider_bis.val)))
915 slider_bis.on_changed(lambda v:neu_zeichnen(int(slider_von.val),int(slider_bis.val)))
916 text_box_start.on_submit(on_submit_start) # Start-Datum abrufen
917 text_box_end.on_submit(on_submit_end)     # Ende-Datum abrufen
918 # -----
919 #      22      Initialisierung Widget-Liste Schaltersperren und History Startzustand
920 # -----
921 # (widget_event-Liste):
922 all_widget_events = [ slider_von, slider_bis, text_box_start, text_box_end, btn_clr,
923                       btn_online, btn_diag1, btn_diag2, btn_diag3, btn_diag4]
924 # Das Klavier (die 26 Schalter) obendrauf:
925 all_widget_events.extend(ticker_klavier)
926 # Logbuch speichert Ur-Zustand beim Start , zB. 0 für Apple)
927 s_anf_init = text_box_start.text_disp.get_text().strip()
928 s_end_init = text_box_end.text_disp.get_text().strip()
929 # Wenn Boxen beim ersten Start leer ist df selbst der Startwert
930 if not s_anf_init:
931     s_anf_init = pd.Timestamp(df['Date'].iloc[0]).strftime('%Y-%m-%d')
932     s_end_init = pd.Timestamp(df['Date'].iloc[-1]).strftime('%Y-%m-%d')
933 # History-Startwert aufzeichnen
934 history_bak.append((0, pd.to_datetime(s_anf_init), pd.to_datetime(s_end_init)))
935 print(f"History Startwert registriert: {s_anf_init}")
936 # -----
937 #      23      Ticker-Oszillograph starten/neu zeichnen und fig,ax nachtragen
938 # -----
939 neu_zeichnen(0, len(df)-1)
940 plt.show()
941

```

Ln: 402 Col: 0